Od początku do celu



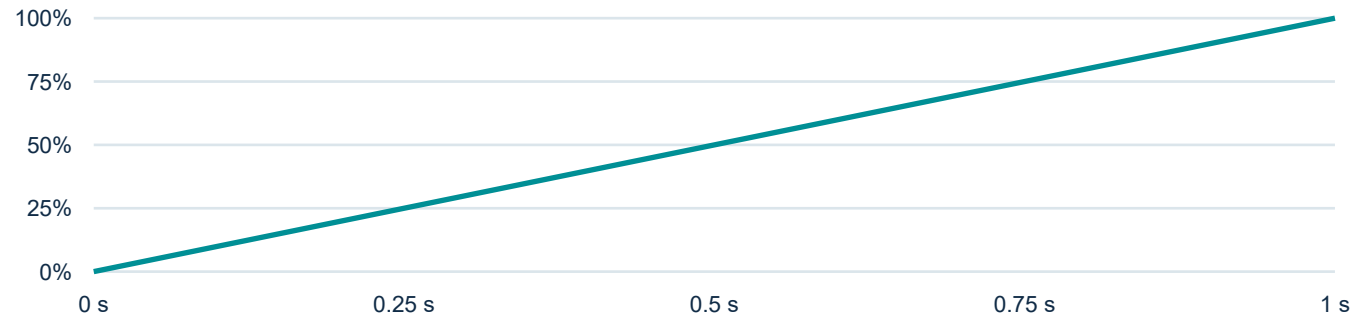
Tween zmienia właściwość obiektu przez określony czas. Postęp czasu to elapsed / duration, od 0 do 1. Easing przekształca ten postęp w część zmiany wartości.

Java Lab udostępnia Tweens.position(), scale(), rotation() i shake(). **PingPong, FadeOut i Pulse w tym materiale są własnymi komponentami**, które dopiero napiszesz. Nie są gotowymi metodami Tweens.

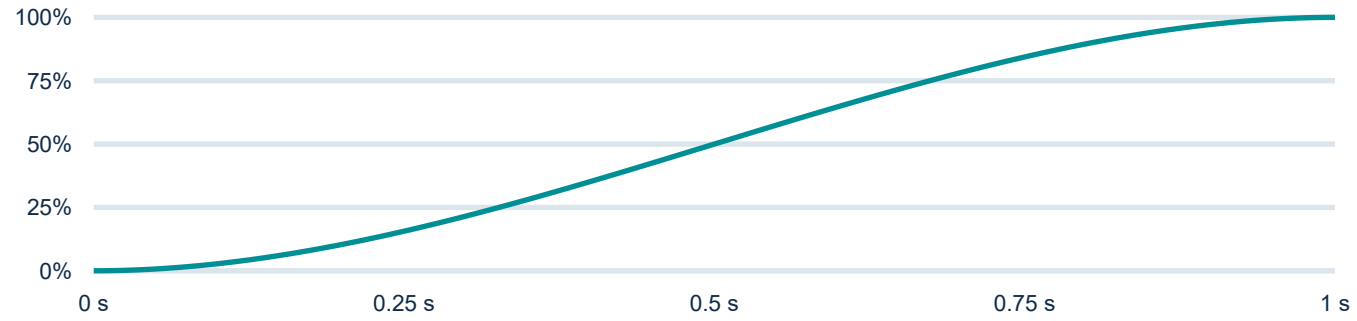
Pozycja i czas to różne wielkości: 50% czasu nie musi oznaczać 50% drogi. Ilustracja pokazuje ideę, a dokładny przebieg odczytuj z wykresów na kolejnych stronach.

Linear i smooth: czytaj wykres

Linear: równomierna zmiana wartości



Smooth: wolniejszy start i zakończenie



Oś X przedstawia czas, a Y część zmiany wartości. Linear: e = t. Smooth: e = t² × (3 − 2t). Nachylenie wykresu pokazuje szybkość zmiany, a nie liczbę klatek.

Przy t = 0,25 linear daje 25%, a smooth 15,625%. Dla ruchu X od 100 do 300 oznacza to odpowiednio X = 150 i X = 131,25. W połowie czasu obie krzywe osiągają połowę zmiany.

Pozycja, skala, obrót i drżenie

Fragment do onCreate(). Ustaw easing jawnie, żeby wynik ćwiczenia nie zależał od wartości domyślnej. W obecnym API Java domyślny Tween.easing ma wartość "linear".

```
GameObject object = createObject("Animated");
object.addComponent(new Sprite(Assets.CHEST, 32, 32));
Tween movement = Tweens.position(object, 300, 100, 1);
movement.easing = "linear";
Tween pulse = Tweens.scale(object, 1.3, 1.3, 0.3);
pulse.easing = "smooth";
Tweens.rotation(object, Math.PI / 2, 0.5);
Tweens.shake(object, 6, 0.2);
```

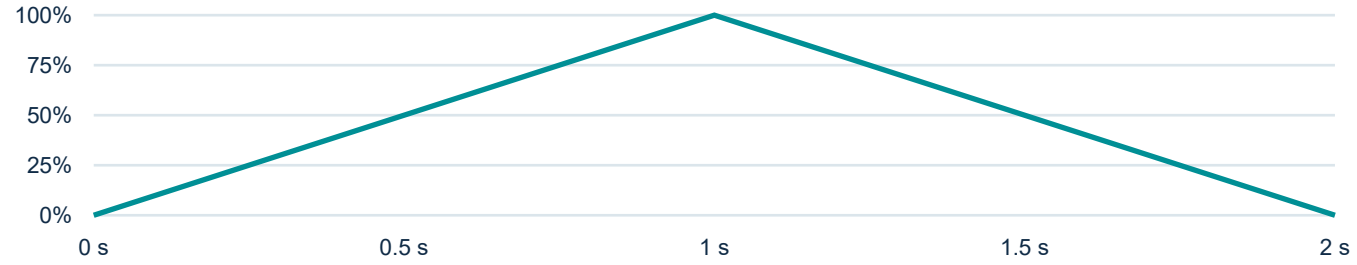
Skala 1,3 to 130% rozmiaru bazowego. Obrót jest w radianach: Math.PI / 2 to 90°. Shake zmienia visualOffset, bez zmiany położenia fizycznego. Skala grafiki nie powiększa automatycznie collidera.

```
Tween movement = Tweens.position(object, 300, 100, 1);
movement.onComplete(() -> System.out.println("Cel osiągnięty"));
// Jeżeli chcesz przerwać animację:
movement.cancel();
```

Naturalne zakończenie wywołuje callback; cancel() nie oznacza completed = true. Nowy tween tej samej właściwości zastępuje poprzedni. Nie twórz tweena bezwarunkowo w każdej aktualizacji.

Tam i z powrotem

Pozycja na drodze A → B → A




```
import engine.*;
public class PingPong extends Component {
    private double a, b;
    private boolean toB = true;
    private Tween tween;
    @Override public void onCreate() {
        a = transform.x; b = a + 200; startLeg();
    }
    private void startLeg() {
        double target = toB ? b : a;
        tween = Tweens.position(gameObject, target, transform.y, 1);
        tween.easing = "linear";
    }
    @Override public void onUpdate(double delta) {
        if (!tween.completed) return;
        toB = !toB; startLeg();
    }
}
```

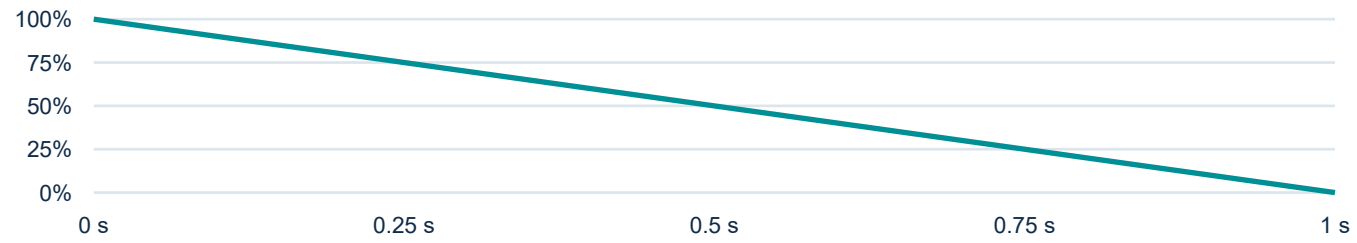
Dodaj new PingPong() do obiektu z rendererem. Jedna droga trwa 1 s, pełny cykl 2 s. To wykres pozycji na odcinku, nie całkowitego postępu czasu. Zmień easing na "smooth" i porównaj zawracanie.

Wygaszanie koloru ShapeRenderer

Nie ma Tweens.fadeOut ani pola Sprite.opacity. Ten własny komponent zmienia kolor ShapeRenderer przez rgba. Nie stosuj go do tekstury Sprite.

```
import engine.*;
public class FadeOut extends Component {
    private double elapsed = 0;
    private double duration = 1;
    private ShapeRenderer renderer;
    @Override public void onCreate() {
        renderer = getComponent(ShapeRenderer.class);
        if (renderer == null)
            throw new IllegalStateException("Brak ShapeRenderer");
    }
    @Override public void onUpdate(double delta) {
        elapsed = Math.min(duration, elapsed + delta);
        double t = elapsed / duration;
        double opacity = 1 - t;
        renderer.color = "rgba(255, 140, 40, " + opacity + ")";
        if (t >= 1) setEnabled(false);
    }
}
```

Przezroczystość maleje: 100% → 0%

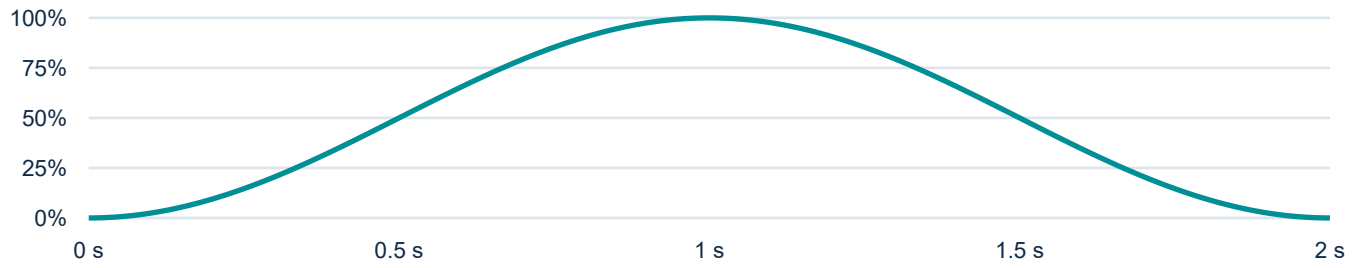


Dodaj new FadeOut() do pomarańczowego kształtu. Niewidoczny obiekt nadal istnieje. Dla jednorazowego efektu możesz na końcu użyć gameObject.destroy(), zamiast tylko wyłączać komponent.

Fadeln i pulsowanie

FadeIn: użyj opacity = t, a początkowy kolor rgba(..., 0) ustaw już w onCreate(). **FadeOut smooth:** oblicz e = t × t × (3 − 2t), potem opacity = 1 − e.

Pulsowanie: minimum → maksimum → minimum




```
import engine.*;
public class Pulse extends Component {
    private double elapsed = 0;
    @Override public void onUpdate(double delta) {
        elapsed += delta;
        double cycle = elapsed / 2;
        double wave = (1 - Math.cos(cycle * Math.PI * 2)) / 2;
        double scale = 1 + 0.2 * wave;
        transform.scale.x = scale;
        transform.scale.y = scale;
    }
}
```

To ręczna animacja cykliczna: skala zmienia się od 1 do 1,2, a pełny cykl trwa 2 s. Kołysanie obrotu możesz obliczyć jako Math.sin(czas × częstotliwość) × amplituda; amplituda jest w radianach.

Zaplanuj animację, potem ją napisz

1. Przesuń kształt z X = 100 do X = 300 w 2 s, porównując linear i smooth.
2. Zmień czas jednego odcinka PingPong na 0,8 s.
3. Wydłuż FadeOut do 2 s.
4. Zmień FadeOut w FadeIn.
5. Zwiększ maksymalną skalę Pulse do 1,4.

Mini-projekt: reakcja skrzynki

W onCreate() przygotuj pole chest wskazujące widoczną skrzynkę. Dodaj poniższy fragment do onUpdate(double delta) sceny. Spacja wyzwala jedno drżenie na nowe naciśnięcie, nie na każdą klatkę przytrzymania.

```
if (input.isKeyPressed("Space")) {
    Tweens.shake(chest, 6, 0.2);
}
```

Pytania i odpowiedzi

Czy 50% czasu zawsze oznacza 50% drogi? **Nie.**
Ile trwa PingPong przy 0,8 s na odcinek? **1,6 s.**
Jaka jest opacity po 0,5 s w FadeOut trwającym 2 s? **0,75.**
Czy cancel() wywołuje onComplete()? **Nie.**
Czy zniknięcie grafiki usuwa obiekt? **Nie.**

Nie łącz dwóch animacji nadpisujących tę samą właściwość. Każdy własny komponent dodawaj do istniejącego obiektu z odpowiednim rendererem. Zachowaj nazwy klas i plików Java.